UNIVERSITI TEKNOLOGI MARA
MARKING SCHEME

COURSE	:	COMPUTER ORGANIZATION AND ARCHITECTURE
COURSE CODE	:	CSC429
EXAMINATION	:	NOVEMBER 2026 (MOCK)
TIME	:	3 HOURS

MARKING SCHEME NOTES

1. This marking scheme accompanies the CSC429 mock paper (November 2026). Generated answers show working. Real-bank questions carry a labelled unofficial AI model answer, not an official scheme.

2. Marking points show the answer and mark allocation. Check model-generated real-bank points against the source paper.

3. Part A = 4 x 5 + 6 x 10; Part B = 1 x 20; total = 100 marks.

4. Generated slots: 2. Real-bank slots: 9. Unofficial model answers:

   9/9.
   For local study use only; model answers are unofficial.

5. Structure from 20 unique CSC429 past papers; every real question remains traceable to its source page.

FOR MARKERS' USE ONLY

This marking scheme consists of 10 printed pages

© Hak Cipta Universiti Teknologi MARA

PART A — MARKING GUIDE

QUESTION A1 (5 marks)

Topic: Instruction formats and addressing modes

Source: CSC429 (1).pdf, page(s) 5

Unofficial model answer — generated by an AI model, not an official UTM marking scheme.

(a) The technique is direct addressing (also called absolute addressing). The instruction's address field A contains the actual/effective memory address of the operand, so the operand is fetched directly from memory location A — no further address calculation is performed.

(b) How it is used: The instruction is fetched and its opcode decoded. The CPU takes the address field A as the effective address (EA = A) and places it on the address bus. The memory is read at location A and the operand is transferred to the CPU (e.g. into a data/accumulator register) for the operation. For example, LOAD A copies the contents of memory word A into the accumulator. Because the address is embedded in the instruction, the operand's location is fixed at assembly/compile time.

(c) Advantage: It is simple and fast — only one memory access is needed to fetch the operand (no extra indirection or address computation), and no special addressing hardware is required. Disadvantage: The address field must be large enough to hold a full memory address, which lengthens the instruction; also the set of addressable operands is limited to the instruction's address-field width, and the code is not position-independent/relocatable.

Unofficial marking guide:

- *1 mark: Correctly names the technique as direct (absolute) addressing.*
- *1 mark: States that the address field A is the effective address of the operand (EA = A).*
- *1 mark: Explains the fetch/execute step — address A is sent to memory and the operand at A is read into the CPU for the operation.*
- *1 mark: Gives ONE valid advantage (e.g. simple/fast, single memory access, no extra address calculation).*
- *1 mark: Gives ONE valid disadvantage (e.g. long address field/instruction, limited addressable range, non-relocatable code).*

Model confidence: high.

Model note: Figure 2 is inferred from the extracted text: an instruction with Opcode + Address A pointing to a memory operand. This matches direct addressing; if the figure showed an extra indirection (address pointing to a pointer), the answer would be indirect addressing. The question numbering references 9(a) but this is slot A1; treated as referring to part (a).

QUESTION A2 (5 marks)

Topic: Parallel and multiprocessor architecture

Source: CSC429 (11).pdf, page(s) 5

Unofficial model answer — generated by an AI model, not an official UTM marking scheme.

Processor organizations with suitable examples

1. Single Instruction, Single Data (SISD) stream. A conventional uniprocessor executes one instruction stream operating on one data stream at a time. *Example:* a basic single-core PC or a simple microcontroller (e.g., an 8051-based embedded controller) executing a sequential program.

2. Single Instruction, Multiple Data (SIMD) stream. One control unit issues the same instruction to many processing elements, each acting on its own data element in lock-step. *Example:* a GPU or a vector processor performing the same arithmetic on an array of pixels (graphics rendering), or Intel MMX/SSE vector instructions adding corresponding elements of two arrays.

3. Multiple Instruction, Single Data (MISD) stream. Several processing elements execute different instructions on the same data stream; it is rare in practice. *Example:* a fault-tolerant/systolic array in which redundant processors apply different operations to the same input data for fault detection, or a pipelined processor where different stages process the same data item.

4. Multiple Instruction, Multiple Data (MIMD) stream. Independent processors execute different instruction streams on different data streams, with each processor having its own control unit. *Example:* a multicore CPU or a cluster of networked workstations running separate threads/processes of a parallel program (e.g., a shared-memory multiprocessor running independent tasks).

*(If the question intends the Flynn-independent classification of processor organizations, the same four categories apply: uniprocessor = SISD, array/vector processor = SIMD, multiprocessor/multicomputer =

© Hak Cipta Universiti Teknologi MARA

MIMD, and the rare MISD case.)*

Unofficial marking guide:

• 1 mark: correct identification and example for SISD (single-core uniprocessor/microcontroller)
• 1 mark: correct identification and example for SIMD (GPU/vector/array processor)
• 1 mark: correct identification and example for MISD (fault-tolerant/systolic/pipelined example)
• 1 mark: correct identification and example for MIMD (multicore/multiprocessor/cluster)
• 1 mark: clear, accurate description of each organization's instruction/data stream behaviour (overall coherence and terminology)
Model confidence: medium.
Model note: The question text does not specify which classification scheme is meant; the answer assumes Flynn's taxonomy (SISD/SIMD/MISD/MIMD), the standard treatment of processor organization in parallel-architecture topics. If the lecturer's notes use a different scheme (e.g., uniprocessor, array processor, multiprocessor, multicomputer), the same examples map onto those categories. No figure or table was supplied.

QUESTION A3 (5 marks)

Topic: Data representation and number systems

Answer: left -43 -> 1 0101011; right 23 -> 0 0010111; -43 - 23 = -66 -> sign-magnitude 1 1000010

• 1 mark for converting both operands to sign-magnitude.
• 1 mark for applying the sign rule for subtraction.
• 1 mark for the correct magnitude operation.
• 1 mark for the correct sign bit.
• 1 mark for the final decimal and byte representation.

QUESTION A4 (5 marks)

Topic: Computer generations and history

Source: CSC429 (8).pdf, page(s) 3

Unofficial model answer — generated by an AI model, not an official UTM marking scheme.

(a) The machines of the zero computer generation (e.g. the abacus, Pascal's Pascaline, Leibniz's stepped reckoner, Babbage's engines) are called mechanical computers because they perform their calculations through physical, moving parts — gears, wheels, levers, rods and ratchets — rather than through electronic components. All arithmetic is carried out by the mechanical movement and positioning of these parts, and the user reads the result from the physical state (e.g. the positions of toothed wheels or dials). There is no electronic circuitry, stored program or electricity involved, so the device is a machine that computes mechanically. [1 mark]

(b) Completed table:

	Mechanical Calculating Machine	Difference Machine
Inventor	Blaise Pascal (1642) — the Pascaline; (Gottfried Wilhelm Leibniz later built the stepped reckoner, 1673)	Charles Babbage (1822, Difference Engine)
Function	Performs basic arithmetic — addition and subtraction (Leibniz's machine also multiplication and division) — automatically by means of gears and wheels, to relieve humans of tedious manual calculation	Automatically computes and tabulates mathematical tables (e.g. polynomial/logarithmic tables) by the method of finite differences, using only addition, and prints the results, eliminating human error in table production

[4 marks: 1 mark per correct inventor, 1 mark per correct function]

Unofficial marking guide:

• 1 mark: Explains that zero-generation machines work through physical/mechanical moving parts (gears, wheels, levers) rather than electronics, so they are called mechanical computers.
• 1 mark: Correct inventor for the Mechanical Calculating Machine (Blaise Pascal / Pascaline; Leibniz's stepped reckoner acceptable).
• 1 mark: Correct function of the Mechanical Calculating Machine (automatic basic arithmetic — addition/subtraction, with multiplication/division on Leibniz's machine).
• 1 mark: Correct inventor for the Difference Machine (Charles Babbage).
• 1 mark: Correct function of the Difference Machine (automatic computation/tabulation of mathematical tables by finite differences, printing results to avoid error).
Model confidence: high.
Model note: The table heading 'Mechanical Calculatin Machine' is an OCR/typo for 'Mechanical Calculating Machine'; treated as

© Hak Cipta Universiti Teknologi MARA

such. The question does not name a specific calculating machine, so Pascal's Pascaline is given as the primary answer with Leibniz's stepped reckoner noted as an acceptable alternative. No figure is required; the table is completed from standard historical knowledge.

QUESTION A5 (10 marks)

Topic: Computer architecture and CPU organisation

Source: CSC429 (6).pdf, page(s) 3

Unofficial model answer — generated by an AI model, not an official UTM marking scheme.

(a) Size of the bus when the word size is 64 bits: The bus size (data bus width) equals the word size, so the bus is 64 bits wide. This means 64 bits (8 bytes) can be transferred in parallel per bus cycle, and the addressable word is 64 bits.

(b) TWO types of bus: (1) Dedicated bus — separate, purpose-specific lines for data and address (e.g., a dedicated data bus and a dedicated address bus). (2) Multiplexed bus — the same lines are shared/time-multiplexed for different functions (e.g., address and data share lines at different times). [Also acceptable as a classification: data bus, address bus and control bus — but the question asks for two types, so dedicated vs multiplexed is the standard pairing.]

(c) Location of the bus between CPU and memory: The system bus (address bus, data bus and control bus) runs between the CPU and main memory. The CPU drives addresses onto the address bus, data travels on the data bus in either direction, and control signals (read/write, memory request) travel on the control bus. Diagrammatically: CPU <—[address bus / data bus / control bus]—> Main Memory, with the same system bus also extended to I/O via the system interconnection. The bus is the shared communication path linking the CPU's bus interface to the memory's bus interface.

(d) System interconnection vs CPU interconnection: System interconnection is the mechanism (the system bus and its arbitration/control logic) that provides communication among all major computer components — CPU, main memory and I/O — i.e., it is the shared, global path between subsystems. CPU interconnection is the internal mechanism within the processor that links the CPU's internal units (registers, ALU, control unit, cache) — e.g., internal processor buses — and connects the CPU to the system bus via its bus interface. In short: system interconnection is external/inter-component (CPU–memory–I/O), whereas CPU interconnection is internal/intra-processor.

(e) Drawback of each type of bus timing: (1) Synchronous bus timing — all transfers are clocked to a fixed clock; the drawback is that the bus must run at the speed of the slowest device, so fast devices are slowed and the bus cannot exploit device speed differences (poor performance/efficiency with mixed-speed devices). (2) Asynchronous bus timing — handshaking (request/acknowledge) is used instead of a clock; the drawback is the extra handshake/control overhead and complexity, which increases transfer latency and cost compared with synchronous timing.

Unofficial marking guide:

- *1 mark: (a) states the bus is 64 bits wide (data bus width = word size).*
- *2 marks: (b) one mark for each correctly named type of bus (dedicated/multiplexed, or data/address/control as accepted).*
- *1 mark: (c) correct diagram/description placing the system bus (address, data, control) between CPU and memory.*
- *2 marks: (d) one mark for correct description of system interconnection (global CPU–memory–I/O path), one mark for correct description of CPU interconnection (internal processor path).*
- *4 marks: (e) two marks for synchronous timing drawback (fixed clock, limited by slowest device) and two marks for asynchronous timing drawback (handshake overhead/complexity/latency).*

Model confidence: high.

Model note: Part (b) is ambiguous in the source: 'types of bus' could mean dedicated vs multiplexed, or data/address/control. Both are accepted in the model answer; the marking guide accepts either classification. Part (c) requires a diagram in the original paper; only a textual illustration can be given here.

QUESTION A6 (10 marks)

Topic: Data representation and number systems

a) Answer: left -39 -> 1 0100111; right -71 -> 1 1000111; -39 - (-71) = 32 -> sign-magnitude 0 0100000

- *1 mark for operand representations.*
- *1 mark for sign handling.*
- *1 mark for magnitude arithmetic.*
- *1 mark for the result sign bit.*
- *1 mark for the final byte.*

© Hak Cipta Universiti Teknologi MARA

b) Answer: left -39 -> 1's 11011000, 2's 11011001; right -71 -> 1's 10111000, 2's 10111001; -39 - (-71) = 32 -> 1's 00100000, 2's 00100000

- *1 mark for the positive operand in binary.*
- *1 mark for the 1's complement of the negative operand.*
- *1 mark for adding 1 to form the 2's complement.*
- *1 mark for the correct 8-bit result.*
- *1 mark for converting the result back to decimal.*

QUESTION A7 (10 marks)

Topic: Memory hierarchy and cache

Source: CSC429 (12).pdf, page(s) 4, 5

Unofficial model answer — generated by an AI model, not an official UTM marking scheme.

(a) The storage devices/memory mentioned in the scenarios are: (i) cassette tape — access method: sequential access (data is read in order from the tape); (ii) floppy disk — access method: direct access (the disk is organised in tracks and sectors, so the head can move directly to a track and then read sequentially within it); (iii) cache memory (implied by Sara's research on larger cache space) — access method: random access (any cache line can be accessed in essentially constant time); (iv) RAM (Hana's high-speed RAM requirement) — access method: random access (any memory location can be accessed directly). Note: if the question intends four distinct devices from the scenarios, the cassette tape, floppy disk, cache, and RAM are the four; the access methods are sequential, direct, random, and random respectively.

(b) SRAM (Static Random Access Memory) stores each bit in a bistable flip-flop (typically six transistors per cell). It retains data as long as power is supplied and does not require periodic refreshing. SRAM is fast, has low access time, but is relatively expensive and has lower density, so it is used for cache memory. DRAM (Dynamic Random Access Memory) stores each bit as charge on a capacitor (one transistor plus one capacitor per cell). The charge leaks away, so DRAM must be refreshed periodically (e.g., every few milliseconds). DRAM is slower than SRAM, but it is cheaper and denser, making it suitable for main memory.

(c) In a three-level cache architecture (L1, L2, L3), the L1 cache is the fastest. It is closest to the CPU core and has the smallest access time. Cache memory is implemented using SRAM (Static RAM) because of its high speed and ability to retain data without refreshing.

Unofficial marking guide:

- *1 mark: (i) cassette tape — sequential access*
- *1 mark: (ii) floppy disk — direct access*
- *1 mark: (iii) cache memory — random access*
- *1 mark: (iv) RAM — random access*
- *2 marks: SRAM description (bistable/flip-flop cell, no refresh, fast, used for cache)*
- *2 marks: DRAM description (capacitor cell, needs refresh, slower, cheaper/denser, used for main memory)*
- *1 mark: L1 cache is the fastest level*
- *1 mark: SRAM is used to implement cache*

Model confidence: high.

Model note: The question text does not explicitly label (i)–(iv); the answer assumes the four devices are cassette tape, floppy disk, cache, and RAM, in the order they appear in the scenarios. If the intended list differs, the access methods should be matched accordingly. The marking guide allocates 4 marks for part (a), 4 for part (b), and 2 for part (c), totaling 10 marks.

QUESTION A8 (10 marks)

Topic: Input/output and interrupts

Source: CSC429 (18).pdf, page(s) 4

Unofficial model answer — generated by an AI model, not an official UTM marking scheme.

(a) Differentiate the involvement of CPU in interrupt-driven I/O and Direct Memory Access (DMA) [4 marks]

In interrupt-driven I/O the CPU is directly involved in every data transfer: the CPU issues the I/O command, then the device transfers one unit (e.g. one byte/word) at a time, and after each transfer raises an interrupt. The CPU must stop its current program, run the interrupt service routine (ISR) to move the data between the device and memory, and then resume. Thus the CPU is tied up for each transfer and pays interrupt overhead per unit, so it is heavily involved and relatively inefficient for large

blocks.
In DMA the CPU is only involved at the start and end: it programs the DMA controller (giving the memory address, device, transfer count and direction) and then continues executing its program. The DMA controller performs the block transfer directly between the device and memory, stealing memory cycles (cycle stealing) without CPU intervention, and interrupts the CPU only once when the whole block is complete. Thus CPU involvement is minimal, giving higher throughput and freeing the CPU for other work.

(b) Define interrupt [2 marks]
An interrupt is a signal (from hardware or software) that causes the CPU to suspend its current instruction sequence, save its state (e.g. PC and registers), and transfer control to a designated interrupt service routine (handler); after the handler finishes, the saved state is restored and the interrupted program resumes. It is used to notify the CPU of asynchronous events such as I/O completion, timers or errors.

(c) Discuss what happens during a context switch [2 marks]
A context switch is the process of saving the state (context) of the currently running process — program counter, registers, status flags and memory-management information — into its process control block (PCB), and then loading the saved context of the next process from its PCB into the CPU registers so that it can resume execution. It is performed by the operating system, typically on an interrupt, a system call, or a scheduling decision, and involves some overhead because no useful user work is done while the switch occurs.

(d) Elaborate on the benefit of a context switch [2 marks]
The benefit is that it enables multiprogramming/multitasking: the CPU can be shared among several processes, so while one process waits for I/O another can use the CPU, improving CPU utilisation and system throughput. It also supports responsiveness (e.g. interactive/time-sharing systems) and allows the OS to enforce protection and fair scheduling by switching between processes safely.

Unofficial marking guide:

• 1 mark: interrupt-driven I/O — CPU issues command and handles each transfer via an ISR, so CPU is involved per unit of data
• 1 mark: interrupt-driven I/O — high CPU overhead/inefficient for large blocks
• 1 mark: DMA — CPU only programs the DMA controller at start and is interrupted once at completion
• 1 mark: DMA — DMA controller transfers block directly to/from memory (cycle stealing), minimal CPU involvement/higher throughput
• 1 mark: interrupt defined as a signal/event that suspends the current program
• 1 mark: CPU saves state and transfers control to an ISR, then restores state and resumes
• 1 mark: context switch saves current process context (PC/registers/PCB)
• 1 mark: context switch loads next process context so it can resume
• 1 mark: benefit — enables multiprogramming/multitasking and better CPU utilisation/throughput
• 1 mark: benefit — supports responsiveness/fair scheduling/protection among processes
Model confidence: high.
Model note: Question text has OCR error 'Direct Manipulation Access'; interpreted conservatively as Direct Memory Access (DMA). No figure or table required.

QUESTION A9 (10 marks)

Topic: Computer architecture and CPU organisation
Source: CSC429 (21).pdf, page(s) 3

Unofficial model answer — generated by an AI model, not an official UTM marking scheme.

(a) The machine cycle is the sequence of steps the CPU performs to fetch, decode, and execute a single instruction, and it is repeated continuously under the control of the control unit. It is normally described in four phases, each with a responsible component:

1. Fetch: The program counter (PC) holds the address of the next instruction. The control unit (CU) places this address on the address bus and issues a memory-read signal; main memory returns the instruction word on the data bus, which is loaded into the instruction register (IR). The PC is then incremented to point to the next instruction. Responsible components: PC, MAR (memory address register), main memory, data bus, IR, and the CU that sequences the transfer.

2. Decode: The control unit examines the opcode held in the IR and determines which operation is required and which operands/registers are involved. The CU translates the opcode into the control signals that drive the datapath. Responsible components: instruction decoder and control unit (with the

© Hak Cipta Universiti Teknologi MARA

IR supplying the opcode).

3. Execute: The arithmetic logic unit (ALU) and the register file carry out the operation. For an arithmetic/logic instruction the ALU computes on operands taken from registers (or from memory via the MAR/MDR), and the result is written back to a register or to the accumulator; for a load/store the memory data register (MDR) and MAR handle the memory transfer; for a branch the PC is updated. Status flags (zero, carry, sign, overflow) in the status/flag register are set. Responsible components: ALU, general-purpose registers/accumulator, MDR, and the CU's control signals.

4. Store/write-back (and interrupt check): The result is written back to the destination register or to main memory, and at the end of the cycle the CU checks for pending interrupts before starting the next fetch. Responsible components: register file, MAR/MDR, main memory, and the interrupt logic within the CU.

Overall, the control unit acts as the coordinator/sequencer, the ALU performs computation, the registers (PC, IR, MAR, MDR, accumulator, flags) hold intermediate state, and main memory supplies instructions and data over the address, data and control buses.

(b) If main memory is corrupted, the fetch phase can return an invalid or altered instruction word into the IR. The control unit then decodes a wrong opcode, so it may issue incorrect control signals, execute the wrong operation, use wrong operands or addresses, and write back a wrong result, or the CPU may halt/trap. Because the PC simply increments, the error can propagate to subsequent instructions, producing incorrect program behaviour or a crash. Detection relies on mechanisms such as parity/ECC, memory-protection faults, or an illegal-opcode/invalid-address trap raised to the operating system; without such checks the corruption goes undetected and the machine cycle produces erroneous results.

Unofficial marking guide:

• 1 mark: states the machine cycle is fetch-decode-execute (with write-back/interrupt check) repeated under control-unit sequencing

• 2 marks: describes fetch phase correctly (PC -> MAR -> memory read -> data bus -> IR, PC incremented) and names PC, MAR, memory, IR

• 2 marks: describes decode phase (CU/instruction decoder interprets opcode in IR and generates control signals)

• 2 marks: describes execute phase (ALU/registers perform operation, flags set; memory access via MAR/MDR for load/store)

• 1 mark: describes write-back/store and interrupt check, and correctly attributes roles to CU, ALU, registers and main memory

• 1 mark: explains that corrupted memory yields a wrong instruction/operand in fetch, leading to wrong decode/execution and propagation of error

• 1 mark: mentions detection/recovery mechanisms (parity/ECC, protection fault, illegal-opcode trap) or that undetected corruption gives wrong results/crash

Model confidence: high.

Model note: Question text is clear; no figure or table required. Marking split (8 + 2) follows the stated subpart marks. This is an unofficial model answer, not an official scheme.

QUESTION A10 (10 marks)

Topic: Input/output and interrupts

Source: CSC429 (4).pdf, page(s) 3

Unofficial model answer — generated by an AI model, not an official UTM marking scheme.

(a) An interrupt is a signal (hardware-generated or software-generated) sent to the CPU that causes it to temporarily suspend the program it is currently executing, save its state (program counter and registers), and transfer control to a special routine called an interrupt service routine (ISR) or interrupt handler. When the ISR finishes, the CPU restores the saved state and resumes the interrupted program as if nothing had happened. In Figure 1 the running program is interrupted by an 'update status' event, which is serviced and then the program continues.

(b) Interrupts are important because they allow the CPU to respond promptly to external and internal events without wasteful polling. Instead of the processor repeatedly checking device status (busy-waiting), a device raises an interrupt only when it needs attention, so the CPU can spend its time on useful work. This gives efficient use of CPU time, fast response to time-critical events (I/O completion, errors, timer ticks), support for multitasking and real-time systems, and better overall system throughput and utilisation.

(c) Two types of interrupt:

1. Hardware (external) interrupt — generated by a device or signal outside the CPU, e.g. a keyboard key press, a disk controller signalling I/O completion, or a timer. It is delivered to the CPU via an interrupt

© Hak Cipta Universiti Teknologi MARA

request line and is asynchronous to the running program.
2. Software (internal/exception) interrupt — generated by the CPU itself as a result of executing an instruction, e.g. a divide-by-zero, an invalid opcode, a page fault, or an explicit trap/system-call instruction. It is synchronous with the program and is used for error handling and operating-system services.
(Other acceptable pairs: maskable vs non-maskable, vectored vs non-vectored, as long as each is explained.)

(d) One option for handling simultaneous interrupts is priority-based interrupt handling (a priority interrupt scheme). Each interrupt source is assigned a priority; when several interrupts arrive at once, the CPU services the highest-priority one first and holds the others pending. A common implementation is a daisy-chain or priority encoder that identifies the highest-priority requesting device, combined with interrupt masking so that lower-priority interrupts cannot pre-empt a higher-priority ISR. Nested interrupts may allow a higher-priority request to interrupt a lower-priority ISR, while lower-priority requests wait until the current ISR completes.

Unofficial marking guide:

• 2 marks: (a) correct definition of interrupt as a signal that suspends the current program, saves state and transfers control to an ISR, then resumes (1 mark for basic idea, 1 mark for save/restore and resume detail).

• 2 marks: (b) importance explained — avoids polling/busy-waiting and gives fast response/efficient CPU use (1 mark for each valid point, up to 2).

• 4 marks: (c) two types explained — 2 marks per type (1 mark naming the type, 1 mark explaining source/example).

• 2 marks: (d) one valid option for simultaneous interrupts discussed, e.g. priority-based handling with masking/nesting (1 mark for naming the option, 1 mark for explaining how it resolves the conflict).

Model confidence: high.

Model note: Figure 1's extracted text is a Windows update error dialog; the answer treats it only as an illustrative interrupt event, as the figure itself cannot be inspected. Part (c) accepts any two correctly explained interrupt types (hardware/software, maskable/non-maskable, vectored/non-vectored).

© Hak Cipta Universiti Teknologi MARA

PART B — MARKING GUIDE

QUESTION B1 (20 marks)

Topic: Assembly-language programming

Source: CSC429 (17).pdf, page(s) 5

Unofficial model answer — generated by an AI model, not an official UTM marking scheme.

Assumption on the equation (OCR correction): Figure 2 reads y = A(A + 2B/C-D 2). The only consistent reading with the available instructions (no exponentiation) is y = A × (A + 2B/C − D²), i.e. the trailing 2 is a superscript square on D. This is assumed throughout; if the figure shows a different expression, the operand order changes but the method is identical.

Sub-expression plan (common to all three programs): compute T1 = 2B/C, T2 = D×D, T3 = A + T1 − T2, then y = A × T3. A, B, C, D are read-only; x, y, z are temporaries; final result in y.

Zero-address program (stack machine):

```
PUSH A        ; stack: A
    PUSH B        ; stack: A, B
    PUSH B        ; stack: A, B, B
    ADD           ; stack: A, 2B       (B+B)
    PUSH C        ; stack: A, 2B, C
    DIV           ; stack: A, 2B/C
    ADD           ; stack: A + 2B/C
    PUSH D        ; stack: (A+2B/C), D
    PUSH D        ; stack: (A+2B/C), D, D
    MUL           ; stack: (A+2B/C), D*D
    SUB           ; stack: A + 2B/C - D*D
    PUSH A        ; stack: (A+2B/C-D*D), A
    MUL           ; stack: A*(A+2B/C-D*D)
    POP y         ; store final answer in y
```

One-address program (accumulator machine):

```
LOAD B        ; ACC = B
    ADD B         ; ACC = 2B
    DIV C         ; ACC = 2B/C
    ADD A         ; ACC = A + 2B/C
    STOR x        ; x = A + 2B/C   (save partial result)
    LOAD D        ; ACC = D
    MUL D         ; ACC = D*D
    STOR z        ; z = D*D
    LOAD x        ; ACC = A + 2B/C
    SUB z         ; ACC = A + 2B/C - D*D
    MUL A         ; ACC = A*(A + 2B/C - D*D)
    STOR y        ; y = final answer
```

Three-address program (register/operand form ADD(p,q,r) meaning p = q op r):

```
ADD (x, B, B)     ; x = 2B
    DIV (x, x, C)     ; x = 2B/C
    ADD (x, A, x)     ; x = A + 2B/C
    MUL (z, D, D)     ; z = D*D
    SUB (x, x, z)     ; x = A + 2B/C - D*D
    MUL (y, A, x)     ; y = A*(A + 2B/C - D*D)  final answer in y
```

Comment on correctness: In every version the read-only registers A, B, C, D are only ever read as source operands, the temporaries x and z hold intermediate values, and the final result is written to y, satisfying the stated constraints.

Unofficial marking guide:

• 4 marks: zero-address program — correct use of PUSH/POP with stack semantics, correct operand order for DIV and SUB, and final POP into y

• 4 marks: one-address program — correct LOAD/STOR/ADD/SUB/MUL/DIV sequence, correct use of temporaries x and z, final STOR y

© Hak Cipta Universiti Teknologi MARA

• 4 marks: three-address program — correct ADD/DIV/MUL/SUB with (p,q,r) destination-first convention, correct final MUL into y

*• 4 marks: correct evaluation of the expression (2B/C, D*D, A + 2B/C − D*D, then multiply by A) in all three programs*

• 2 marks: comments and correct assembly notation throughout, with A, B, C, D unmodified and final answer in y

• 2 marks: overall clarity, consistent register/temporary usage and no redundant or missing operations

Model confidence: medium.

Model note: The equation in Figure 2 is OCR-ambiguous ('D 2'); assumed to be D squared since no exponent instruction exists. The three-address table shows 'MUL (p.q.r)' — read as MUL(p,q,r). The (p,q,r) convention is assumed to be destination-first (p = q op r); if the lecturer uses source-first ordering, swap the operand positions accordingly. Figure 2 itself was not available beyond the extracted text.

END OF MARKING SCHEME

© Hak Cipta Universiti Teknologi MARA